MyNotesAssist

RAG-powered Study Assistant — Project Documentation, Architecture & Interview Prep

Role focus	Full-Stack AI Engineer
Backend	Python · Flask · LangChain · FAISS · OpenAI / Gemini
Frontend	React (CRA) · Axios
Web fallback	DuckDuckGo Search + BeautifulSoup (whitelisted study sites)
Storage	Local FAISS index + uploaded PDFs on disk; localStorage for chat history
Auth	Single-user HMAC-signed token, 24h expiry

1. Project Overview

MyNotesAssist is a full-stack **Retrieval-Augmented Generation (RAG)** study companion. A user uploads PDFs of their college notes, asks questions, and gets answers grounded in those documents — with source citations, exam-style answer modes, and a graceful fallback to trusted study sites and general knowledge when the notes don't cover a topic.

Why this project exists

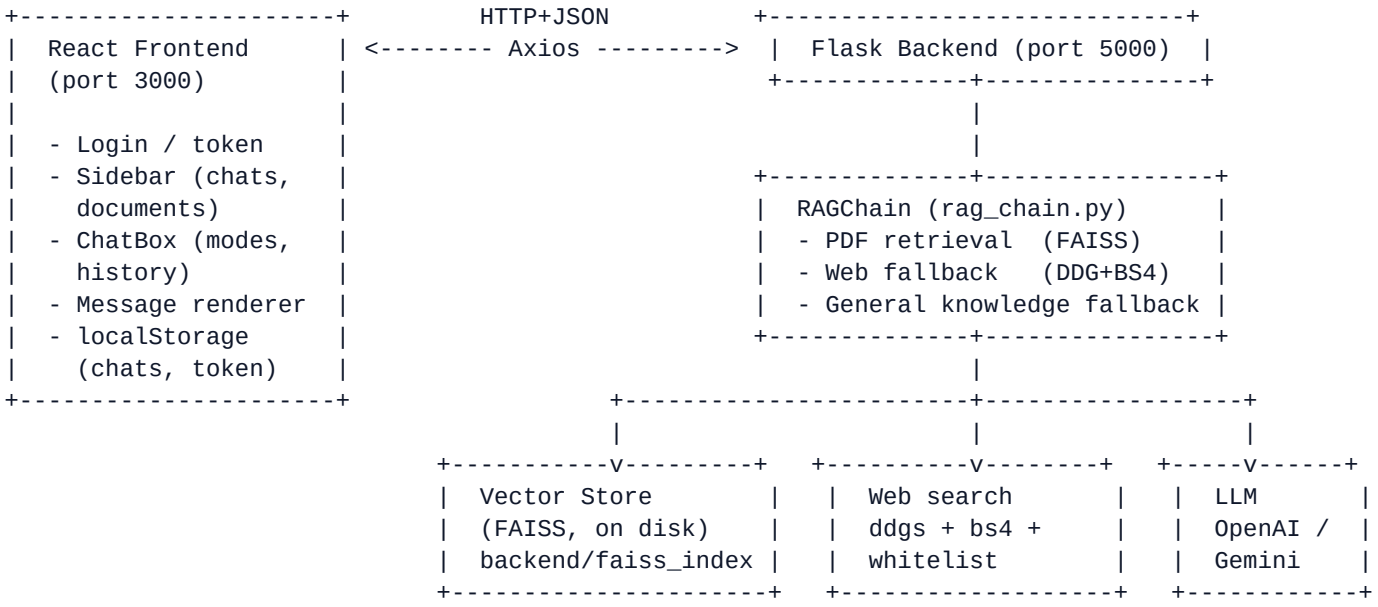
Pure chatbots hallucinate; pure document search doesn't synthesize. RAG combines the precision of vector retrieval with the fluency of an LLM. Students get exam-ready answers tied to their own materials.

Key capabilities

- Upload PDFs → chunked → embedded → indexed in FAISS.
- Ask in 6 modes: **Auto, 1 mark, 2 marks, Long answer, Seminar, Presentation** — each with a tailored prompt.
- Three-tier fallback: **PDF** → **trusted web sites** → **general knowledge**. Never refuses.
- Per-document tools: Summarise, Notes, Quiz.
- Multi-chat sidebar with rename / delete / persistence in localStorage.
- Auto-greeting on every new chat; understands casual conversation, not just academic queries.
- Source citations on every answer (PDF page numbers or clickable web URLs).

2. High-Level Architecture

The app is a classic two-tier React + Flask split, with the AI/retrieval pipeline encapsulated in the backend.




```
Uploaded PDFs:  backend/uploads/    (filesystem persistence)
FAISS index:    backend/faiss_index/  index.faiss + index.pkl
Chat history:   browser localStorage (key: mynotesassist.chats)
```

3. Tech Stack — and Why Each Choice

Layer	Tech	Why this choice
Web framework	Flask 3	Lightweight, single-file friendly, perfect for a small AI service. FastAPI
LLM orchestration	LangChain	Provides PDF loaders, text splitters, vector store wrappers, and provid
Vector DB	FAISS (CPU)	Library, not a server — runs in-process, no extra infra, persists to two
Embeddings	OpenAI text-embedding-3-small or Google gemini-embedding-001	Cheap, accurate, 1536/3072-dim. 3-small model is ~5× cheaper than -la
Chat LLM	gpt-4o-mini or gemini-flash-latest	Fast, cheap, supports markdown/code generation. Sufficient for acade
PDF parsing	pypdf via PyPDFLoader	Pure Python, no native deps. Good enough for text-based PDFs; wou
Chunking	RecursiveCharacterTextSplitter (1000 chars, 150 overlap)	Splits on natural boundaries (paragraph → sentence → space) so chu
Web search	ddgs (DuckDuckGo) + BeautifulSoup4 + requests	DDG is free, no API key, allows site-restriction. BS4 strips boilerplate.
Auth	HMAC-signed token (custom)	Single-user app didn't need a full JWT/OAuth setup. HMAC + base64
Frontend	React 18 (CRA) + Axios	Minimal cognitive overhead; CRA proxy forwards /api calls to Flask in
Chat persistence	localStorage (browser)	No backend DB needed for per-user chat — keeps the app stateless f

4. End-to-End Flow (Request → Response)

4.1 Upload phase

```
[User clicks Upload PDF]
     ↓
POST /upload  (multipart, file)
     ↓
app.py: save to backend/uploads/<filename>
     ↓
document_loader.load_pdf(path, filename)
     - PyPDFLoader → list of Documents (one per page)
     - RecursiveCharacterTextSplitter (chunk_size=1000, overlap=150)
     - tag each chunk with metadata: source=<filename>, page=<n+1>
     ↓
embeddings.get_embeddings()       → OpenAIEmbeddings or Gemini
vector_store.add_documents(chunks)
     - FAISS.from_documents() if first ever, else .add_documents()
     - save_local() → backend/faiss_index/index.faiss  +  index.pkl
     ↓
Response: { success: true, filename, chunks: <count> }
```

4.2 Ask phase (the heart of the app)

```
[User types question + picks mode]
     ↓
POST /ask  { query, mode }
     ↓
app.py → rag_chain.answer(query, mode)
     ↓
-----------------------------------------------------------------
Step 1.  detect_chat()
     - If query is a greeting/thanks/meta → use CHAT_PROMPT (small talk)
     - Returns immediately, no retrieval.
-----------------------------------------------------------------
Step 2.  detect_mode()
     - If mode == "auto": parse keywords ("1 mark", "long", "seminar"...)
     - Picks formatting rules from MODE_RULES (1mark / 2marks / long / seminar / presentation / au
-----------------------------------------------------------------
Step 3.  PDF retrieval
     - vector_store.similarity_search(query, k=6 or 10)
     - FAISS computes cosine sim between query embedding and chunk embeddings.
     - Build prompt with PDF_HEADER + MODE_RULES + retrieved chunks + question.
     - LLM responds.
     - If response matches NOT_AVAILABLE_PATTERNS  →  fall through.
     - Else return: { answer, sources, mode, source_kind: "pdf" }
-----------------------------------------------------------------
Step 4.  Web fallback
     - web_search.web_search_docs(query)
       · ddgs.text(<query> site:geeksforgeeks.org OR site:javatpoint.com OR ...)
       · For top 3 hits inside whitelist:
            requests.get(url) → BeautifulSoup → strip script/nav/footer
            keep first ~4500 chars
     - Build prompt with WEB_HEADER + MODE_RULES + scraped pages + question.
     - LLM responds.
     - Return: { answer, sources, mode, source_kind: "web" }
-----------------------------------------------------------------
Step 5.  General-knowledge fallback (final layer)
     - Build prompt with GENERAL_HEADER + MODE_RULES + question (no context).
```

```
- LLM answers from its own training data.
- For casual messages it replies in 1-3 sentences.
- Return: { answer, sources: [], source_kind: "general" }
```

4.3 Frontend handling

The Message component renders markdown manually (no heavy library). It splits out fenced code blocks first so triple-backtick contents aren't reinterpreted as markdown, then handles headings, bullets, tables, bold, inline code, links, and bare URLs. Source pills are clickable when web URLs are present (target=_blank).

5. File-by-File Walkthrough

5.1 Backend

backend/app.py — Flask entry point

• Boots Flask, configures CORS, mounts routes.

• Loads .env (override=True) so swapping providers needs only a restart.

• Routes: **/login, /upload, /ask, /documents, /document (DELETE), /summarize, /notes, /quiz, /health**.

• _check_auth() validates the Bearer token via auth.verify_token() before every protected route.

• On boot creates the singleton VectorStoreManager and RAGChain (in-memory FAISS handle).

backend/auth.py — single-user HMAC auth

• verify_credentials() compares APP_USERNAME / APP_PASSWORD using hmac.compare_digest (constant-time).

• create_token() = base64(payload) + '.' + HMAC-SHA256(secret, payload). Payload contains username + epoch.

• verify_token() recomputes the HMAC; rejects if mismatch or older than 24h.

• Why HMAC and not JWT? Single-user app — no need for claims, audiences, or library deps.

backend/document_loader.py — PDF → chunks

• Uses LangChain's PyPDFLoader (pypdf under the hood) to extract text per page.

• RecursiveCharacterTextSplitter with separators ['\n\n', '\n', '. ', ' ', ''] — falls back through coarser splits when sentences are too long.

• chunk_size=1000 chars, overlap=150 — keeps ~250 tokens per chunk, with overlap to preserve context across boundaries.

• Normalises page numbers from 0-indexed (pypdf) to 1-indexed (human-friendly).

backend/embeddings.py — provider switch

• Single function get_embeddings() returns either OpenAIEmbeddings (text-embedding-3-small) or GoogleGenerativeAIEmbeddings.

• Provider is read from LLM_PROVIDER env var. Switching providers requires deleting backend/faiss_index/ — embeddings from different models are not comparable.

backend/vector_store.py — FAISS persistence wrapper

• _load() reads backend/faiss_index/index.faiss on startup (allow_dangerous_deserialization=True is needed because FAISS uses pickle for the docstore).

• add_documents() builds the index lazily on first PDF, then incrementally adds for subsequent uploads.

• delete_document() walks the docstore, finds chunk IDs whose source matches the filename, and deletes.

• list_documents() / get_document_chunks() — used by the Sidebar and per-document tools.

• save_local() writes two files: index.faiss (raw vectors + structures) and index.pkl (metadata + docstore).

backend/rag_chain.py — the RAG brain

• Defines MODE_RULES (per-mode formatting prompts) and three header prompts: PDF_HEADER, WEB_HEADER, GENERAL_HEADER.

• _is_not_available() detects the 'NOT_IN_PDF' sentinel or natural-language refusals, triggering the next fallback.

• answer() implements the 5-step flow described in §4.2.

• summarize() / generate_notes() / generate_quiz() — bulk operations over a single document, tied to sidebar buttons.

• _invoke() coerces multimodal LangChain message-content (str | list[parts]) into a clean string — important for Gemini which sometimes returns parts arrays.

backend/web_search.py — fallback fetcher

• ALLOWED_SITES whitelist: geeksforgeeks, javatpoint, tutorialspoint, programiz, wikipedia, w3schools, scaler.

• _ddgs_search() builds a 'site:a OR site:b OR ...' DuckDuckGo query.

• _fetch_text() pulls the page with a real-browser User-Agent, BS4 strips script/style/nav/header/footer/aside/form/noscript, normalises whitespace, caps at 4500 chars.

• Returns LangChain Documents tagged page='web' so RAGChain can build a uniform prompt.

• Hardened: every step is wrapped in try/except so a flaky network or a renamed CSS class on GfG never crashes the pipeline.

5.2 Frontend

frontend/src/api.js — Axios layer

• Single axios instance with baseURL from REACT_APP_API_URL or http://localhost:5000.

• On module load, re-attaches the token from localStorage so refreshes don't kick the user out.

• Tiny exported helpers: login, upload, ask, listDocuments, deleteDocument, summarize, generateNotes, generateQuiz.

frontend/src/chats.js — multi-chat hook

• Custom React hook useChats() managing the full chats array + activeId.

• Persists to localStorage under *mynotesassist.chats*; auto-migrates older keys (*mynotemaker.chats*, *mynotemaker.chat.history*).

• Public API: chats, activeChat, newChat, selectChat, renameChat, deleteChat, updateMessages.

• Auto-names a chat from the first user message (truncated to 40 chars).

frontend/src/App.js — root + token gate

• If no token in localStorage → render Login.

• Otherwise instantiate useChats() and pass props to Sidebar + ChatBox.

frontend/src/components/Login.js

• Single form: username + password → POST /login → store token in localStorage and Axios default header.

frontend/src/components/Sidebar.js

• Two sections: **Chats** (with + New, rename via ✎ or double-click, delete via ✕) and **Documents** (Summarise / Notes / Quiz / Delete buttons per file).

• Document actions dispatch a custom 'ai-message' DOM event consumed by ChatBox — decouples sidebar from chat state.

frontend/src/components/ChatBox.js

• Reads messages from activeChat; calls updateMessages() to append.

• Mode selector at the top right + + New chat button.

• Editable chat title (double-click to rename inline).

• If activeChat has zero messages, renders a static greeting bubble — friendly without paying for an LLM call.

frontend/src/components/Message.js

• Hand-rolled markdown renderer (deliberately tiny — no react-markdown dep).

• Pulls fenced code blocks out first, then handles headings, bullets, tables, bold, inline code, links, bare URLs.

• Source pills: PDF pills are static, web pills are clickable anchor tags with ■ prefix.

frontend/src/components/FileUpload.js

• Hidden <*input type=file*> + visible button. POSTs multipart/form-data to /upload, then calls onUploaded() to refresh the documents list.

6. How Each Model / Component Works

6.1 Embedding model

An **embedding model** maps a string of text to a fixed-dimensional vector (1536 for OpenAI text-embedding-3-small, ~3072 for Gemini gemini-embedding-001). Semantically similar texts land close together in vector space (small cosine distance). Models are trained with contrastive objectives — pull paraphrases together, push unrelated pairs apart.

In MyNotesAssist, every PDF chunk is embedded once at upload time and stored in FAISS. At query time, the user's question is embedded and compared against all chunk vectors using cosine similarity (FAISS uses inner-product on L2-normalised vectors, which is equivalent to cosine).

6.2 FAISS — Facebook AI Similarity Search

• **What it is**: a C++ library (with Python bindings) for fast nearest-neighbour search over dense vectors. Authored at Meta.

• **Why it's fast**: supports indexing strategies — FlatL2 (exact, brute force), IVF (inverted file with cell partitioning), HNSW (graph-based), PQ (product quantization for memory compression).

• **Index used here**: LangChain's default is IndexFlatL2 — brute force but exact. Fine for <100K chunks (covers years of notes).

• **Persistence**: save_local() writes two files — index.faiss (the raw vectors and ANN structures) and index.pkl (LangChain's docstore mapping vector_id → original Document with metadata).

• **Why FAISS over alternatives**: ChromaDB is heavier and bundles its own server; Pinecone/Weaviate are SaaS and add latency + cost; pgvector requires Postgres. FAISS runs in-process, has zero infra, and can be migrated later if the dataset grows.

6.3 Chat LLM

gpt-4o-mini (default) or gemini-flash-latest. Both are instruction-tuned, support tool/JSON calling, are fast (sub-second TTFT), and cheap (~$0.15/1M input tokens). Temperature is set to 0.3 — low enough for factual academic answers, high enough for natural prose.

6.4 Retrieval-Augmented Generation pipeline

```
1. Question  →  embedding (1536-d vector)
2. FAISS top-k similar chunks  (k=6 normally, k=10 for long/seminar/presentation)
3. Build prompt:
       SYSTEM_HEADER
       MODE_RULES
       === CONTEXT ===
       [chunk 1 with source/page tag]
       ---
       [chunk 2 ...]
       === QUESTION ===
       <user query>
4. LLM generates answer following the mode's format.
5. Sources extracted from chunk metadata → returned alongside answer.
```

6.5 Web fallback model

• **ddgs (DuckDuckGo Search)**: scrapes DDG's HTML SERP. No API key needed. Site-restricted query keeps results inside a study-friendly whitelist.

• **BeautifulSoup4**: HTML parser. Strips boilerplate (script, style, nav, header, footer, aside, form, noscript), prefers <article> / <main> over <body> when present, normalises whitespace.

• **requests**: simple HTTP GET with a real-browser User-Agent and 8s timeout to avoid hangs.

7. Storage & Persistence

7.1 Where uploaded PDFs go

Every uploaded file is saved to **backend/uploads/<original-filename>** before processing. The file lives there as a real file on disk for the lifetime of the project (until you delete it via the UI or rm the folder).

7.2 Where the FAISS index lives

backend/faiss_index/ contains two files after the first upload:

- **index.faiss** — the raw vector data (binary).
- **index.pkl** — Python pickle of LangChain's InMemoryDocstore mapping each vector to the original chunk text + metadata (source filename, page number).

On every save_local() call, both files are rewritten atomically. On every Flask boot, _load() reads them back into memory.

7.3 Will my documents survive a session?

Yes — fully. Both the original PDF (in uploads/) and the indexed embeddings (in faiss_index/) are on the filesystem. Closing the browser, killing the Flask process, or rebooting the machine doesn't lose anything. They only disappear if you:

- Click Delete on the file in the sidebar (removes from FAISS + uploads/).
- Manually delete the backend/uploads/ or backend/faiss_index/ folders.
- Switch embedding providers (e.g. OpenAI → Gemini) — embeddings produced by different models are not compatible. The README warns to delete faiss_index/ and re-upload after the swap.

7.4 What about chat history?

Chat history (chats array, messages, names) lives entirely in **browser localStorage** under the key **mynotesassist.chats**. It survives page refresh and browser restart, but is per-browser and per-machine — clearing site data wipes it. There is intentionally no server-side store for chats — keeps the backend stateless and avoids privacy concerns for a single-user student app.

7.5 What database?

None traditional. The project uses three flat-file persistences:

What	Where	Format
PDFs	backend/uploads/	Original PDF files
Vector index	backend/faiss_index/index.faiss	FAISS binary
Chunk metadata	backend/faiss_index/index.pkl	Pickled docstore
Chat history	browser localStorage	JSON string
Auth token	browser localStorage + Authorization head	base64.payload + HMAC

If the project ever needed to support multiple users or 1M+ chunks, the migration path would be: chats → Postgres/Mongo, FAISS → pgvector or Pinecone or a self-hosted Milvus, files → S3.

8. Why FAISS — Deep Dive

FAISS (Facebook AI Similarity Search) is the most-cited open-source library for approximate nearest-neighbour (ANN) search over dense vectors. Three reasons it fits this project:

a) It's a library, not a server

FAISS runs in the same Python process as Flask. No extra Docker container, no port to expose, no auth to configure. For a project meant to run on a student's laptop, this is the lowest-friction option.

b) Disk-based persistence, in-memory speed

save_local() / load_local() serialise the whole index to two files. At startup the files are mmap'd back into RAM. Searches are sub-millisecond at the scales we care about (a few thousand chunks).

c) Mature, well-tuned, exact and approximate

FAISS supports many index types — IndexFlatL2 (exact), IndexIVFFlat (cell-clustered, fast approximate), IndexHNSWFlat (graph-based, very fast), IndexIVFPQ (compressed for billions of vectors). LangChain defaults to the simplest exact index, perfect when recall matters more than latency at this scale.

Comparison with alternatives

Option	Pros	Cons	Verdict for this project
FAISS	Library, fast, mature, free	No built-in metadata filtering; needs LangChain wrapper	Chosen ✓
Chroma	Easy SDK, includes metadata	Bundles a server, slower in pure Python ops	Overkill
Pinecone	Managed, scales infinitely	SaaS cost + latency + API key	Wrong for offline use
pgvector	SQL + filtering on rows	Requires Postgres install + extension	Heavy
Weaviate / Milvus	Production-grade	Need a Docker container + tuning	Premature

9. Security, Limits & Production Considerations

9.1 What is implemented

- All routes except /login and /health require a Bearer token.
- HMAC-SHA256 signed token with 24h expiry; constant-time compare using hmac.compare_digest.
- PDF-only upload validation by extension.
- FAISS deserialisation flag (allow_dangerous_deserialization=True) is acceptable here because we control the index files; would NOT be acceptable if files came from outside.

9.2 What I would harden for production

- Replace HMAC with proper JWT + rotation, add refresh tokens, store APP_SECRET in a vault.
- Add rate limiting (Flask-Limiter) on /ask and /upload.
- Validate PDF magic bytes, not just extension; cap upload size.
- Run web_search.py inside a small fetch service with caching (avoid hammering DDG).
- Add request-level structured logging (request id, user, latency, source_kind, mode).
- Move from CORS '*' to an allow-list of frontend origins.
- Containerise (Dockerfile per service); deploy backend behind gunicorn + nginx; serve frontend as static build through CloudFront / nginx.
- Move FAISS to pgvector or Milvus once chunks > ~500K; store PDFs on S3.

10. Interview Q&A; — Full-Stack AI Engineer

All answers tied to *this* project where applicable. Read out loud — they're written to be spoken.

10.1 RAG fundamentals

Q. What is RAG and why use it instead of fine-tuning?

RAG — Retrieval-Augmented Generation — grounds an LLM's answer in documents fetched at query time. You embed your documents, store the vectors, and at query time fetch the top-k most similar chunks and stuff them into the prompt as context. Compared with fine-tuning, RAG is cheaper (no training), updates are instant (just re-embed new docs), and citations are natural (you know which chunk produced the answer). Fine-tuning is for behavior/style, RAG is for knowledge.

Q. Walk me through what happens when I ask a question in this app.

Five steps. (1) detect_chat — if it's a greeting, route to a small-talk prompt and return. (2) detect_mode — pick formatting rules from the user's mode or keyword (1mark / 2marks / long / seminar / presentation / auto). (3) FAISS similarity search returns top-k chunks; build a prompt with PDF_HEADER + mode rules + context + question; if the LLM doesn't say NOT_IN_PDF, return. (4) Otherwise call DuckDuckGo restricted to a whitelist of study sites, scrape the top 3 with BeautifulSoup, build a WEB_HEADER prompt, return. (5) If web also fails, fall back to a general-knowledge prompt with no context — the assistant answers from the model's own knowledge so it never refuses.

Q. Why three fallback layers and not just one?

PDFs first because the user's notes are the source of truth for their exam. Web second because trusted study sites — GeeksforGeeks, JavaTPoint, TutorialsPoint — are accurate enough for a student and free to scrape. General knowledge last because refusing every off-topic question is a terrible user experience for a study companion. Sources are tagged on the response so the user always knows where the answer came from.

Q. How is chunking done and why those parameters?

RecursiveCharacterTextSplitter with chunk_size=1000 chars, overlap=150. The recursive splitter tries paragraph breaks first, then sentences, then words, then characters — so chunks stay semantically coherent rather than cutting mid-sentence. 1000 chars is roughly 250 tokens, small enough to fit many chunks in the prompt's context window, big enough to carry meaning. The 150-char overlap means a fact straddling two chunks isn't lost — the question's embedding will retrieve at least one chunk that contains the full fact.

Q. How does similarity search actually work?

Both the question and every chunk are embedded into the same vector space (1536-d for OpenAI). FAISS stores the chunk vectors and computes distances at query time — by default L2 distance, which on L2-normalised vectors is monotonically equivalent to cosine similarity. The library returns the indices of the k smallest distances. LangChain's docstore then maps those indices back to the original Document objects so we get the page_content + metadata.

Q. Why FAISS and not Chroma, Pinecone, or pgvector?

FAISS is a library, not a server — it runs in-process, persists to two flat files, and has zero infra cost. Chroma bundles a server and is overkill at this scale. Pinecone is a managed SaaS and adds latency, cost, and an API key — wrong fit for an offline student tool. pgvector requires installing Postgres and an

extension. For a few thousand chunks, FAISS Flat is exact, sub-millisecond, and free. If we hit 1M chunks the migration target would be Milvus or pgvector.

10.2 LangChain & embeddings

Q. Why use LangChain at all? Couldn't you call OpenAI directly?

You could, but you'd be writing a lot of glue. LangChain gives you: a uniform LLM/embedding interface across providers (one env var swaps OpenAI ↔ Gemini), PDF loaders, text splitters, vector store wrappers (FAISS / Chroma / pgvector), and Document/metadata abstractions. We use exactly those pieces and skip the heavier 'agent' / 'chain' abstractions because our pipeline is straightforward.

Q. What's an embedding and how is it trained?

An embedding is a fixed-length vector that represents the semantic meaning of a piece of text. Models are trained with contrastive learning: pull paraphrases or related sentences together in vector space, push unrelated pairs apart. OpenAI's text-embedding-3-small is 1536-d. The result: cosine similarity between two embeddings approximates how 'meaningfully similar' the two texts are.

Q. text-embedding-3-small vs -large — which one and why?

Small. It's about a fifth of the cost and the recall on student-style notes (one-paragraph chunks of a single topic) is virtually identical. -large is worth it when you have heterogeneous, long-tail content where the extra dimensions capture finer distinctions.

Q. If a user switches from OpenAI to Gemini after indexing, what happens?

Their queries will be embedded by Gemini but the chunks were embedded by OpenAI. Different embedding spaces — distances are meaningless. The README warns to delete backend/faiss_index/ and re-upload. We could also detect provider change at boot and refuse to load an index built with a different provider.

10.3 Frontend & full-stack

Q. Why localStorage for chat history instead of a backend table?

It's a single-user app and chats are personal. Storing them server-side would mean adding a database, user identity beyond the simple HMAC token, sync logic, and privacy concerns. localStorage gives instant reads/writes, survives refresh and reboot, and keeps the backend stateless. Tradeoffs: no cross-device sync, lost if site data is cleared, and a 5-10MB quota per origin (more than enough for thousands of messages).

Q. How does the multi-chat sidebar avoid stale state?

All chat state lives in a single useChats() hook in App.js. The hook owns the chats array + activeId and persists to localStorage on every state change. Sidebar and ChatBox receive props from App; they never read localStorage directly. This single source of truth means renaming a chat in the sidebar instantly updates the title in the chat header, with no event bus.

Q. Why CRA and not Vite or Next.js?

Inertia and ecosystem familiarity. CRA's dev proxy makes /api forwarding to Flask trivial. Vite would be faster and lighter but the migration cost wasn't worth it for a small project. Next.js is overkill — there's no SSR or routing requirement.

Q. How do you stop CORS from biting you?

Flask-CORS with origins='*' for dev. In production I'd lock that down to the actual frontend origin. The Authorization header is sent on every request via Axios's default header (set once at module load from localStorage).

Q. How is the Bearer token sent and validated?

On login the backend returns a base64(payload) + '.' + HMAC-SHA256(secret, payload). The frontend stores it in localStorage and Axios's default Authorization header. Every protected route calls _check_auth() which calls auth.verify_token() — that splits on '.', re-pads the base64, recomputes the HMAC with hmac.compare_digest, and verifies the timestamp is within 24h.

Q. Why HMAC and not JWT?

Functionally similar — both are signed tokens. JWT adds a standard header, claims, and a much larger library surface. For a single-user app I didn't need the standard. Migration path: switch to PyJWT, keep the same secret, change the encode/decode functions. Same threat model, more interoperability.

10.4 System design & scale

Q. How would you scale this to 10,000 users?

(1) Auth: replace HMAC with JWT + refresh tokens + a users table. (2) Backend: containerise, run gunicorn behind nginx, scale horizontally. (3) Vector store: migrate to pgvector or Milvus — FAISS in-process won't share state across replicas. (4) PDFs: move to S3. (5) Cache web fallback responses (DDG is rate-limited). (6) Add a CDN for the React build. (7) Embedding calls: batch and queue — they're the slowest part of upload.

Q. Where's the bottleneck at upload time?

Embedding API calls. A 50-page PDF produces ~100-150 chunks; embedding them is one network round-trip per batch. text-embedding-3-small accepts batches of 2048 inputs per request, so we should batch instead of sequential. LangChain does some batching already; for big PDFs I'd add explicit batching and async.

Q. Where's the bottleneck at query time?

Almost always the LLM call (1-3 seconds). FAISS search is sub-millisecond. Web fallback adds 3-8 seconds (DDG search + 3 parallel fetches). Optimisations: stream the LLM response, cache embeddings of common queries, parallelise the web scrape.

Q. How would you handle hallucination?

Three layers in this app: (1) PDF prompt explicitly says 'use ONLY the provided context' and emit NOT_IN_PDF if you can't. (2) Web prompt cites source titles. (3) General fallback explicitly tells the LLM to say it doesn't know if it doesn't. On top: lower temperature (0.3), smaller answer length on 1mark mode, and source pills so the user can verify.

Q. How would you evaluate this RAG pipeline?

Build a test set of (question, expected-answer-from-doc) pairs. For each, run the pipeline and check: (a) was the right chunk retrieved (recall@k)? (b) did the LLM use the chunk faithfully (faithfulness)? (c) is the final answer correct (accuracy)? Tools: LangSmith, RAGAS, or a hand-rolled eval harness. The 'NOT_IN_PDF' sentinel makes it easy to measure recall — if it fires when the answer was in the doc, retrieval is at fault.

Q. How would you support PDFs with images, tables, or scans?

Switch the loader. PyPDFLoader is text-only. Use unstructured.io or pdfplumber for tables, plus an OCR step (Tesseract or vision-LLM) for scanned pages. Tables can be serialised to markdown and embedded as a single chunk. Images can be captioned by a vision model and the captions embedded alongside text.

10.5 Tricky / behavioural

Q. What was the hardest design decision in this project?

Making the assistant feel useful when the PDF doesn't have the answer. A pure RAG bot says 'not available'. ChatGPT just answers but ignores the user's notes. The three-layer fallback — PDF first, then whitelisted web, then general knowledge with a casual-tone branch for small talk — was the design that made the app feel alive without crossing into making things up.

Q. If you had a week to ship v2, what would you build?

Streaming answers (token-by-token), multi-PDF tagging so a question can be scoped to one folder, OCR for scanned PDFs, and per-user accounts with chats synced to Postgres. Then memory of past chats so 'remind me what we said about CNNs last week' works.

Q. What did you learn building this?

Three things. (1) RAG quality lives or dies on chunking — bad chunks make great LLMs look stupid. (2) Prompt engineering > model choice for structured outputs; the right system prompt with a small, cheap model beats a big model with a vague prompt. (3) The web fallback was scope creep that turned out to be the most-used path — never assume the happy case is the common case.

Appendix — Quick reference

Endpoints

```
POST  /login             {username, password}        →  {success, token}
POST  /upload            multipart file (pdf)         →  {success, filename, chunks}
POST  /ask               {query, mode}                →  {answer, sources, mode, source_kind}
GET   /documents                                      →  {documents: [{filename, chunks}]}
DELETE /document?filename=X                           →  {success}
POST  /summarize         {filename}                   →  {answer, sources, mode}
POST  /notes             {filename}                   →  {answer, sources, mode}
POST  /quiz              {filename}                   →  {answer, sources, mode}
GET   /health                                         →  {status: ok}
```

Modes (formatting rules)

Mode	What you get
Auto	Detects from query keywords; default for plain questions.
1 Mark	Answer + one-line justification; picks options for MCQs.
2 Marks	Definition / Explanation / Example / Applications / Adv-Disadv.
Long Answer	800-1200 word essay with code, flow chart, table, conclusion.
Seminar	Title, abstract, objectives, content, examples, Q&A.
Presentation	Slide-by-slide outline with speaker notes.

Project layout

```
MyNotesAssist/
■■■ backend/
■   ■■■ app.py                # Flask routes
■   ■■■ auth.py               # HMAC token
■   ■■■ document_loader.py    # PDF → chunks
■   ■■■ embeddings.py         # OpenAI / Gemini embeddings
■   ■■■ vector_store.py       # FAISS persistence
■   ■■■ rag_chain.py          # 3-tier RAG pipeline
■   ■■■ web_search.py         # DDG + BS4 fallback
■   ■■■ requirements.txt
■   ■■■ .env.example
■   ■■■ uploads/              # uploaded PDFs (persisted)
■   ■■■ faiss_index/          # index.faiss + index.pkl (persisted)
■■■ frontend/
■   ■■■ package.json
■   ■■■ public/index.html
■   ■■■ src/
■       ■■■ index.js
■       ■■■ App.js
■       ■■■ App.css
■       ■■■ api.js
■       ■■■ chats.js          # multi-chat hook + localStorage
■       ■■■ components/
■           ■■■ Login.js
■           ■■■ Sidebar.js    # chats + documents
■           ■■■ ChatBox.js    # chat UI + greeting bubble
■           ■■■ Message.js    # markdown / code / table renderer
■           ■■■ FileUpload.js
■■■ README.md
■■■ MyNotesAssist_Documentation.pdf  ← this file
```